Лабораторная работа №4.
Язык XML и правила оформления XML-документов. Программная обработка документов HTML или XML

Цель работы: изучить основные конструкции языка XML, правила оформления XML-документов, методы контроля содержимого документа с помощью схем; изучение основ применения языков для автоматизации процесса разметки и добавления интерактивных возможностей веб-страниц.

Теоретический материал

Язык XML

XML разрабатывался как язык с простым формальным синтаксисом, удобный для создания и обработки документов программами и одновременно удобный для чтения и создания документов человеком, с подчёркиванием нацеленности на использование в Интернете. Язык называется расширяемым, поскольку он не фиксирует разметку, используемую в документах: разработчик волен создать разметку в соответствии с потребностями к конкретной области, будучи ограниченным лишь синтаксическими правилами языка. Расширение XML – это конкретная грамматика, созданная на базе XML и представленная словарём тегов и их атрибутов, а также набором правил, определяющих какие атрибуты и элементы могут входить в состав других элементов. Сочетание простого формального синтаксиса, удобства для человека, расширяемости, а также базирование на кодировках Юникод для представления содержания документов привело к широкому использованию как собственно XML, так и множества производных специализированных языков на базе XML в самых разнообразных программных средствах.

XML документ можно представить в виде древовидной структуры, которая начинается с «корня» и заканчивается «листьями».

Ниже приведен пример XML документа:

```
<?xml version="1.0" encoding="Windows-1251"?>
  <mail>
    <to>user1@domain.ru</to>
    <from>user2@domain.ru </from>
    <subject>Встреча</heading>
    <body>Позвони мне завтра утром</body>
  </mail>
```

В первой строке помещена XML декларация, описывающая версию XML и используемую кодировку. В следующей строке описан *корневой* элемент документа. В последующих четырех строках описаны 4 *дочерних* элемента относительно корневого: (*to, from, subject* и *body*). И в последней строке указывается *конец корневого* элемента.

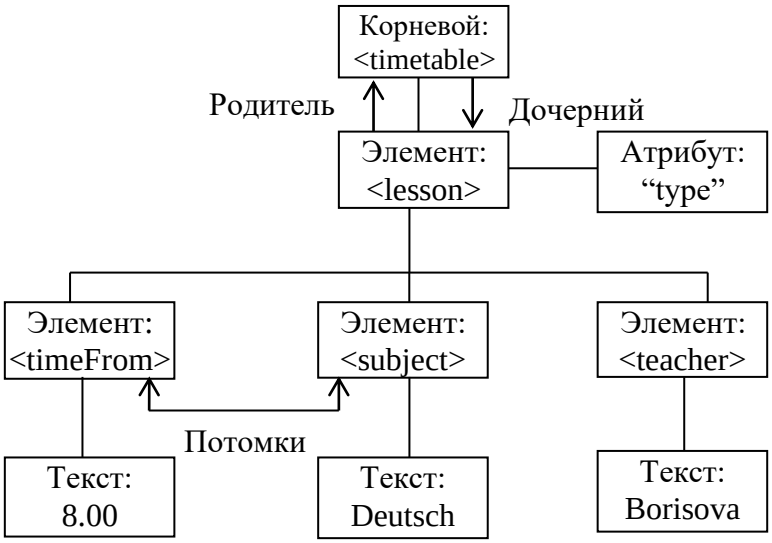
XML документ должен содержать *корневой* элемент, который является *родительским* для всех остальных элементов. Любой элемент (кроме

находящихся на самом нижнем уровне дерева) может иметь вложенные элементы (дочерние элементы).

С помощью терминов *родитель*, *дочерний* и *потомок* описываются отношения между элементами в дереве XML документа. *Родители* содержат *дочерние элементы*, а дочерние элементы одного уровня называются *потомками* (братьями или сестрами).

Элементы могут иметь *текстовое содержимое* и *атрибуты*.

Ниже приведена схема, показывающая отношения между элементами и атрибутами этого документа:



Кроме того в XML документе следует выполнять еще несколько несложных правил:

- Каждый XML элемент иметь закрывающий тэг.
- XML тэги чувствительны к регистру.
- Должны соблюдаться правила вложенности элементов.
- XML документы должны иметь корневой элемент, который является родительским для всех других элементов.
- Значения XML атрибутов должны заключаться в двойные кавычки.

Некоторые из символов (их всего 5) имеют специальное значение в XML. Для их обозначения используются альтернативные сочетания:

Замена	Символ
<	<
>	>
&	&
'	'
"	"

Комментарии в XML выделяются также как и в HTML:

<!-- Это комментарий -->

В отличие от HTML, который игнорирует повторы пробелов, XML сохраняет полную длину строки из пробелов.

XML документы, оформленные в соответствии с приведенными выше правилами, называются “правильно оформленными”. Если при этом они еще и соответствуют правилам DTD (Document Type Definition), то они являются и “действительными”.

Действительные XML документы содержат ссылку на DTD файл, например:

<!DOCTYPE note SYSTEM "mail.dtd">

DTD схема предназначена для определения структуры XML документа путем задания списка допустимых элементов, например

```
<!DOCTYPE mail [
  <!ELEMENT note (to,from,subject,body)>
  <!ELEMENT to       (#PCDATA)>
  <!ELEMENT from     (#PCDATA)>
  <!ELEMENT subject (#PCDATA)>
  <!ELEMENT body    (#PCDATA)>
]>
```

Консорциум W3C поддерживает альтернативную к *DTD*, основанную на XML схему, называемую как “*XML Schema*”:

```
<xs:element name="mail">
   <xs:complexType>
    <xs:sequence>
      <xs:element name="to"       type="xs:string"/>
      <xs:element name="from"    type="xs:string"/>
      <xs:element name="subject" type="xs:string"/>
      <xs:element name="body"    type="xs:string"/>
    </xs:sequence>
   </xs:complexType>
</xs:element>
```

Для проверки действительности XML документа можно использовать специальные валидаторы, например *W3C валидатор* (http://validator.w3.org/).

Согласно спецификации W3C XML программа должна прекратить обработку XML документа, как только будет обнаружена ошибка в этом документе.

DTD (Document Type Definition) определяет допустимые строительные блоки XML документа, путем указания списка допустимых элементов и атрибутов.

DTD может описываться как внутри XML документа, так и с помощью внешней ссылки.

Пример внутреннего описания:

```
<?xml version="1.0"?>
<!DOCTYPE note [
  <!ELEMENT mail (to,from,subject,body)>
  <!ELEMENT to       (#PCDATA)>
  <!ELEMENT from    (#PCDATA)>
```

```
  <!ELEMENT subject (#PCDATA)>
  <!ELEMENT body    (#PCDATA)>
]>
<note>
  <to>user1@domain.ru</to>
  <from>user2@domain.ru</from>
  <heading>Встреча</heading>
  <body> Позвони мне завтра утром </body>
</note>
```

В данном примере:

- **!DOCTYPE mail** определяет корневой элемент документа **mail.**
- **!ELEMENT note** определяет элемент **note,** который содержит четыре элемента: "*to, from, subject, body*".
- **!ELEMENT to** определяет элемент **to** типа "#PCDATA".
- **!ELEMENT from** определяет элемент **from** типа "#PCDATA".
- **!ELEMENT subject** определяет элемент **subject** типа "#PCDATA".
- **!ELEMENT body** определяет элемент **body** типа "#PCDATA".

Пример внешнего описания:

```
<?xml version="1.0"?>
 <!DOCTYPE note SYSTEM "mail.dtd">
    <note>
      <to>user1@domain.ru</to>
      <from>user2@domain.ru</from>
      <heading>Встреча</heading>
      <body> Позвони мне завтра утром </body>
    </note>
```

С помощью DTD XML файлы могут содержать описание собственного формата. Независимые группы людей могут обмениваться данными. DTD-схема может быть использована для проверки действительности, как документов получаемых извне, так и собственных документов.

С точки зрения DTD все XML документы (HTML документы) строятся из следующих строительных блоков:

- *Элементы* (Elements). Элементы могут содержать текст, другие элементы или быть пустыми.
- *Атрибуты* (Attributes). Атрибуты всегда размещаются внутри открывающего тэга элемента. Атрибуты всегда записываются в виде пары *имя/значение*.
- *Entities*.
- *PCDATA*. Означает структурированные символьные данные. Подлежит анализу с помощью парсера на наличие специальных символов (entities) и элементов разметки.
- *CDATA*. Означает текст, содержимое которого не рассматривается анализатором.

При описании структуры элементов, содержащих вложенные элементы можно использовать специальные операторы, задающие множественность их вхождения:

- ‘**+**’: один и более раз, например, *<!ELEMENT note (message+)>*
- ‘*****’: ноль и более раз, например, *<!ELEMENT note (message*)>*
- ‘**?**’: ноль и один раз, например, *<!ELEMENT note (message?)>*

Объявление вида *<!ELEMENT note (message|body)>* указывает на возможность вхождения в элемент *note* другого элемента: *message* ИЛИ *body*.

Атрибуты элементов описываются в форме:

<!ATTLIST element-name attribute-name attribute-type default-value>

Например, **<!ATTLIST payment type CDATA "check">** соответствует XML коду: *<payment type="check" />*.

Тип атрибута может принимать одно из значений:

Тип	Описание
CDATA	Символьные данные
(*en1*|*en2*|..)	Значение из списка
ID	Уникальный *id*
IDREF	*id* другого элемента
IDREFS	Список других *id*
NMTOKEN	Допустимое XML имя
NMTOKENS	Список допустимых XML имен
ENTITY	Специальные символы
ENTITIES	Список специальных символов
NOTATION	Имя нотации
xml:	Предопределенное XML имя

Значение по умолчанию может быть одним из следующих:

Значение	Интерпретация
#REQUIRED	Обязательно
#IMPLIED	Не обязательно
#FIXED *value*	Значение фиксировано

Так же как DTD схема XML Schema определяет допустимые строительные блоки XML документа:

- элементы, входящие в документ
- атрибуты, допустимые в документе
- дочерние элементы
- порядок дочерних элементов
- количество дочерних элементов
- может ли элемент быть пустым или может содержать текст
- типы элементов и атрибутов
- фиксированные и значения по умолчанию элементов и атрибутов

Предполагается, что в перспективе DTD схемы будут замещены XML Schema в большинстве веб-приложений поскольку XML Schema:

- Расширяема для будущих добавлений.

- Более богатые и мощные выразительные возможности.
- Является реализацией XML.
- Поддерживает типы данных.
- Поддерживают пространства имен.

XML Schema стала W3C рекомендацией в 2001 году. Рассмотрим в качестве примера XML документ:

```
<?xml version="1.0" encoding="Windows-1251"?>
 <mail>
    <to>user1@domain.ru</to>
    <from>user2@domain.ru </from>
    <subject>Встреча</heading>
    <body>Позвони мне завтра утром</body>
 </mail>
```

Структура данного документа может быть рассмотрена с помощью следующей *XML Schema*:

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
targetNamespace="http://www.myhp.edu"
xmlns=" http://www.myhp.edu"
elementFormDefault="qualified">
<xs:element name="mail">
   <xs:complexType>
    <xs:sequence>
       <xs:element name="to" type="xs:string"/>
       <xs:element name="from" type="xs:string"/>
       <xs:element name="subject" type="xs:string"/>
       <xs:element name="body" type="xs:string"/>
    </xs:sequence>
   </xs:complexType>
</xs:element>
</xs:schema>
```

В данном примере элемент *mail* имеет тип **complexType**, поскольку содержит другие элементы. Другие элементы документа имеют простой тип, поскольку не включают других элементов.

Ссылка на схему в XML документе выглядит следующим образом:

```
<?xml version="1.0"?>
   <mail
    xmlns="http://www.myhp.edu"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.myhp.edu mail.xsd">
      <to>user1@domain.ru</to>
      <from>user2@domain.ru</from>
      <subject>Встреча</heading>
      <body>Позвони мне завтра утром</body>
   </mail>
```

Элемент <schema> является корневым элементом любой схемы XML Schema.

Данный элемент может содержать несколько атрибутов, например:

```
<?xml version="1.0"?>
  <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://www.myhp.edu"
  xmlns="http://www.myhp.edu"
  elementFormDefault="qualified">
  ...
  ...
  </xs:schema>
```

Следующий фрагмент: xmlns:xs=http://www.w3.org/2001/XMLSchema указывает на то, что элементы и типы данных, используемые в схеме входят в пространство *http://www.w3.org/2001/XMLSchema*. Причем, все элементы и типы данных из этого пространства имен должны иметь префикс **xs.**

Фрагмент: targetNamespace=http://www.myhp.edu указывает на то, что элементы, определяемые в схеме входят в пространство “http://www.myhp.edu”.

Во фрагменте: xmlns=http://www.myhp.edu указывается, что пространством имен по умолчанию является http://www.myhp.edu. Следующий фрагмент: elementFormDefault="qualified" указывает на то, что любые элементы, объявленные в схеме, должны принадлежать пространству имен.

Ссылка на внешнюю схему может выглядеть следующим образом:

```
<?xml version="1.0"?>
  <mail xmlns=" http://www.myhp.edu "
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation=" http://www.myhp.edu mail.xsd">
      <to>user1@domain.ru</to>
      <from>user2@domain.ru </from>
      <subject>Встреча</heading>
      <body>Позвони мне завтра утром</body>
  </mail>
```

Здесь строка xsi:schemaLocation="http://www.myhp.edu mail.xsd"> указывает на местоположение файла схемы.

Синтаксис для определения простого элемента:

```
<xs:element name="xxx" type="yyy"/>
```

где *xxx* – имя элемента и *yyy* тип данных элемента.

Встроенными типами данных элементов являются следующие:

- xs:string
- xs:decimal
- xs:integer
- xs:boolean
- xs:date
- xs:time

Например, фрагмент XML-документа:

<lastname>Refsnes</lastname>

<age>36</age>

<dateborn>1970-03-27</dateborn>

описывается в схеме следующим образом:

<xs:element name="lastname" type="xs:string"/>

<xs:element name="age" type="xs:integer"/>

<xs:element name="dateborn" type="xs:date"/>

Следующие фрагменты:

<xs:element name="color" type="xs:string" default="red"/>

и

<xs:element name="color" type="xs:string" fixed="red"/>

описывают значение элемента по умолчанию и фиксированное значения соответственно.

Все атрибуты описываются простыми типами данных.

Простые элементы не могут иметь атрибутов. Если элемент имеет атрибуты, он рассматривается как имеющий сложный тип. Сам атрибут рассматривается всегда как имеющий простой тип.

Описание атрибута обычно дается следующим образом:

<xs:attribute name="xxx" type="yyy"/>

где *xxx* – имя атрибута, а *yyy* - определяет тип данных атрибута.

Встроенными типами данных для атрибутов являются соледующие:

- xs:string
- xs:decimal
- xs:integer
- xs:boolean
- xs:date
- xs:time

Например, XML элемент с атрибутом:

<lastname lang="EN">Smith</lastname>

описывается соответствующей схемой:

<xs:attribute name="lang" type="xs:string"/>

Значение по умолчанию и фиксированное значение атрибута описываются следующим образом:

<xs:attribute name="lang" type="xs:string" default="EN"/>

и

<xs:attribute name="lang" type="xs:string" fixed="EN"/>

соответственно.

Для описания обязательных атрибутов (по умолчанию все атрибуты являются необязательными) используется следующая нотация:

<xs:attribute name="lang" type="xs:string" use="required"/>

Для задания допустимых значений величин XML элементов и атрибутов можно использовать ограничения. Ограничения на значения XML элементов называются *фасетами*.

Список возможных ограничений приводится ниже:

Ограничение	Описание
enumeration	Определяет список допустимых значений
fractionDigits	Указывет максимальное число десятичных позиций. Должно быть неотрицательным
length	Указывает точное число символов или элементов в списке. Должно быть неотрицательным
maxExclusive	Указывает верхнюю границу числовых значений
maxInclusive	Указывает верхнюю границу числовых значений
maxLength	Указывает максимальное число символов или элементов в списке. Должно быть неотрицательным
minExclusive	Указывает нижнюю не включаемую границу числовых значений
minInclusive	Указывает нижнюю включаемую границу числовых значений
minLength	Указывает минимальное число символов или элементов в списке. Должно быть неотрицательным
pattern	Определяет точно последовательность символов (шаблон), которая допустима
totalDigits	Определяет точное число цифр. Должно быть неотрицательным
whiteSpace	Указывает как обрабатывать неотображаемые символы (пробел, табуляция и др.)

Примеры ограничения:

```
<xs:element name="age">
  <xs:simpleType>
    <xs:restriction base="xs:integer">
      <xs:minInclusive value="0"/>
      <xs:maxInclusive value="120"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>

<xs:element name="password">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-zA-Z0-9]{8}"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

При описании документов, имеющих сложную иерархическую структуру, можно сначала определить все элементы и атрибуты, а затем ссылаться на них.

Рассмотрим следующий пример:

```
<employee>
```

```
  <firstname>John</firstname>
  <lastname>Smith</lastname>
</employee>
```

Сложный элемент в схеме можно определить следующим образом:

```
<xs:element name="firstname" type="xs:string"/>
<xs:element name="lastname" type="xs:string"/>
<xs:element name="employee">
    <xs:complexType>
        <xs:sequence>
            <xs:element ref="firstname"/>
            <xs:element ref="lastname"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>
```

Другой подход заключается в повторном использовании именованных элементов.

1. Элемент "*employee*" может быть объявлен непосредственно по имени:

```
<xs:element name="employee">
 <xs:complexType>
  <xs:sequence>
   <xs:element name="firstname" type="xs:string"/>
   <xs:element name="lastname" type="xs:string"/>
  </xs:sequence>
 </xs:complexType>
</xs:element>
```

Поскольку дочерние элементы "*firstname*" и "*lastname*" окружены *индикатором* <sequence>, то они должны в документе появляться именно в этой последовательности.

2. Элемент "*employee*" может иметь атрибут типа, ссылающийся на имя сложного типа:

```
<xs:element name="employee" type="personinfo"/>
<xs:complexType name="personinfo">
 <xs:sequence>
  <xs:element name="firstname" type="xs:string"/>
  <xs:element name="lastname" type="xs:string"/>
 </xs:sequence>
</xs:complexType>
```

Несколько элементов могут ссылаться на тот же самый сложный тип:

```
<xs:element name="employee" type="personinfo"/>
<xs:element name="student" type="personinfo"/>
<xs:element name="member" type="personinfo"/>

<xs:complexType name="personinfo">
 <xs:sequence>
```

```
    <xs:element name="firstname" type="xs:string"/>
    <xs:element name="lastname" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

Также можно строить сложный элемент на базе существующего сложного элемента с добавлением нескольких элементов:

```
<xs:element name="employee" type="fullpersoninfo"/>

<xs:complexType name="personinfo">
  <xs:sequence>
    <xs:element name="firstname" type="xs:string"/>
    <xs:element name="lastname" type="xs:string"/>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="fullpersoninfo">
  <xs:complexContent>
    <xs:extension base="personinfo">
      <xs:sequence>
        <xs:element name="address" type="xs:string"/>
        <xs:element name="city" type="xs:string"/>
        <xs:element name="country" type="xs:string"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Пустой сложный элемент не может иметь содержимого, только атрибуты.

С помощью специальных индикаторов можно указывать, как элементы могут использоваться в документах.

Всего используется семь индикаторов:

- Индикатор порядка (*Order*):
  - All
  - Choice
  - Sequence
- Индикатор вхождения (*Occurrence*):
  - maxOccurs
  - minOccurs
- Групповые (*Group*) индикаторы:
  - Group name
  - attributeGroup name

Пример:

```
<xs:element name="person">
  <xs:complexType>
    <xs:sequence>
```

```
    <xs:element name="full_name" type="xs:string"/>
    <xs:element name="child_name" type="xs:string"
    maxOccurs="10"/>
   </xs:sequence>
  </xs:complexType>
</xs:element>
```

Программная обработка документов HTML или XML

DOM (Document Object Model) – представляет собой стандарт консорциума W3C для программного доступа к документам HTML или XML. Фактически это платформо- и языково-нейтральный интерфейс, позволяющий программам и сценариям динамически обращаться и обновлять содержимое, структуру и стиль документа.

В рамках данного стандарта можно выделить 3 части:

- Core DOM – стандартная модель любого структурированного документа
- XML DOM - стандартная модель XML документа
- HTML DOM - стандартная модель HTML документа

DOM определяет объекты и свойства всех элементов документа и методы (интерфейс) для доступа к ним.

HTML DOM определяет объекты и свойства всех HTML элементов и методы (интерфейс) для доступа к ним. Иначе говоря, HTML DOM описывает каким образом необходимо получать, изменять, добавлять и удалять HTML элементы.

В соответствии с моделью DOM все, что содержится внутри HTML документа - является узлом. То есть HTML документ представляется в виде дерева узлов, которыми являются элементы, атрибуты и текст.

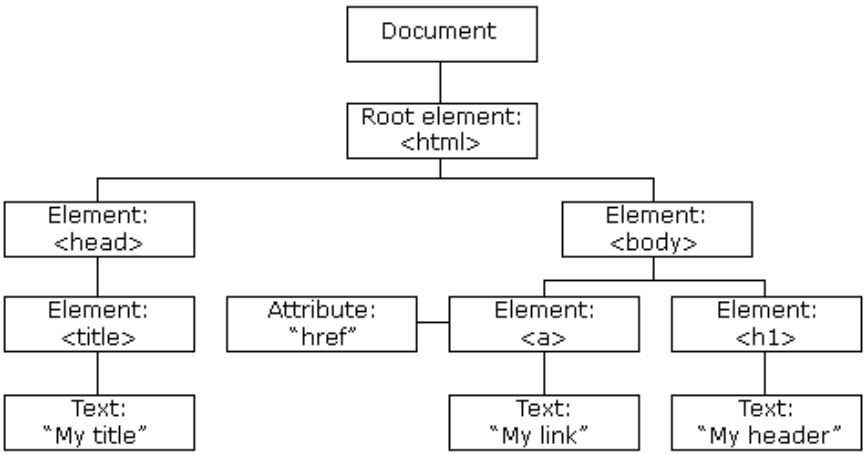


Рисунок 1. Узлы дерева HTML документа

Согласно модели DOM:

- Весь документ представляется узлом документа;
- Каждый HTML тэг является узлом элемента;
- Текст внутри HTML элементов представляется текстовыми узлами;
- Каждому HTML атрибуту соответствует узел атрибута;
- Комментарии являются узлами комментариев.

Рассмотрим следующий пример.

```
<html>
  <head>
    <title>HTML документ</title>
  </head>
  <body>
    <h1>Заголовок </h1>
    <p>Просто текст</p>
  </body>
</html>
```

В этом примере корневым узлом является тэг <html>. Все остальные узлы содержатся внутри <html>. У этого узла имеется два дочерних узла: <head> и <body>. Узел <head> содержит узел <title>, а узел <body> содержит узлы <h1> и <p>.

Следует обратить особое внимание на то, что текст, расположенный в узле элемента соответствует текстовому узлу. В примере ***<title>HTML документ</title>*** узел элемента <title> содержит текстовый узел «*HTML документ*», то есть «*HTML документ*» не является значением элемента <title>. Тем не менее, в рамках HTML DOM значение текстового узла может быть доступно посредством свойства *innerHTML*.

Все узлы HTML документа могут быть доступны посредством дерева, при этом их содержимое может быть изменено или удалено, а также можно добавить новые элементы.

Все узлы дерева находятся в иерархических отношениях между собой. Для описания этих отношений используются термины *родитель*, *дочерний элемент* и *потомок*. Родительские узлы имеют дочерние узлы, а дочерние элементы одного уровня называются потомками (братьями или сестрами).

В отношении узлов дерева соблюдаются следующие принципы:

- Самый верхний узел дерева называется корневым;
- Каждый узел, за исключением корневого, имеет ровно один родительский узел;
- Узел может иметь любое число дочерних узлов;
- Конечный узел дерева не имеет дочерних узлов;
- Потомки имеют общего родителя.

В рамках DOM модели HTML можно рассматривать как множество узловых *объектов*. Доступ к ним осуществляется с помощью *JavaScript* или других языков программирования. Программный интерфейс DOM включает в себя набор стандартных *свойств* и *методов*.

Свойства представляют некоторые сущности (например, <h1>), а *методы* - действия над ними (например, *добавить <a>*).

К типичным свойствам DOM относятся следующие:

- x.*innerHTML* – внутреннее текстовое значение HTML элемента **x**;
- x.*nodeName* – имя **x**;
- x.*nodeValue* – значение **x**;
- x.*parentNode* – родительский узел для **x**;

- x.*childNodes* – дочерний узел для **x**;
- x.*attributes* – узлы атрибутов **x.**

Узловой объект, соответствующий HTML элементу поддерживает следующие методы:

- x.*getElementById*(id) – получить элемент с указанным **id**;
- x.*getElementsByTagName*(name) – получить все элементы с указанным именем тэга (**name**);
- x.*appendChild*(node) – вставить дочерний узел для **x**;
- x.*removeChild*(node) – удалить дочерний узел для **x**.

Для получения текста из элемента <*p*> со значением атрибута *id* "*demo*" в HTML документе можно использовать следующий код:

txt = document.getElementById("demo").innerHTML

Тот же самый результат может быть получен по-другому:

txt=document.getElementById("demo").childNodes[0].nodeValue

В рамках DOM возможны 3 способа доступа к узлам:

1. С помощью метода *getElementById*(ID). При этом возвращается элемент с указанным **ID**.
2. С помощью метода *getElementsByTagName*(name). При этом возвращаются все узлы с указанным именем тэга (в виде индексированного списка). Первый элемент в списке имеет нулевой индекс.
3. Путем перемещения по дереву с использованием отношений между узлами.

Для определения длины списка узлов используется свойство *length*.

```
x = document.getElementsByTagName("p");
for (i = 0; i < x.length; i++) {
        document.write(x[i].innerHTML);
        document.write("<br/>");
}
```

В данном примере внутрь HTML документа вставляется в виде списка текстовое содержимое всех элементов соответствующих тэгу <p>.

Для навигации по дереву в ближайших окрестностях текущего узла можно использовать следующие свойства:

- *parentNode;*
- *firstChild;*
- *lastChild*.

Для непосредственного доступа к тэгам можно использовать 2 специальных свойства:

- document.*documentElement* – для доступа к корневому узлу документа;
- document.*body* – для доступа к тэгу <*body*>.

В HTML DOM каждый узел является объектом, который может иметь методы (функции) и свойства. Наиболее важными являются следующие свойства:

- *nodeName*;
- *nodeValue*;

- *nodeType*.

Свойство *nodeName* указывает на имя узла. Это свойство имеет следующие особенности:

- Свойство *nodeName* предназначено только для чтения;
- Свойство *nodeName* узла элемента точно соответствует имени тэга;
- Свойство *nodeName* узла атрибута соответствует имени атрибута;
- Свойство *nodeName* текстового узла всегда равно **#text**
- Свойство *nodeName* узла документа всегда равно **#document**

Свойство *nodeName* всегда содержит имя тэга HTML элемента в верхнем регистре.

Свойство *nodeValue* указывает на значение узла. Это свойство имеет следующие особенности:

- Свойство *nodeValue* узла элемента не определено;
- Свойство *nodeValue* текстового узла указывает на сам текст;
- Свойство *nodeValue* узла атрибута указывает на значение атрибута.

Свойство *nodeType* возвращает тип узла. Это свойство предназначено только для чтения:

Наиболее важными типами узлов являются следующие:

Тип элемента	Тип узла
Element	1
Attribute	2
Text	3
Comment	8
Document	9

HTML элементы могут быть изменены с посредством использования JavaScript, HTML DOM и событий.

В примере 5 показано, как можно динамически изменять текстовое содержимое тэга <p>:

```
<html>
  <body>
     <p id="p1">Hello World!</p>
     <script type="text/javascript">
          document.getElementById("p1").innerHTML="New text!";
     </script>
  </body>
</html>
```

В JavaScript поддерживается работа со следующими диалоговыми элементами интерфейса:

1) **Alert**. Применяется для уведомления пользователя, работающего с веб-браузером.

Синтаксис: *alert("сообщение");*

2) **Confirm**. Применяется для выбора пользователем одного из двух вариантов ответа «Да/Нет». Соответственно *Confirm* возвращает значение *true/false*.

Синтаксис: *confirm("вопрос");*

3) **Prompt**. Применяется для ввода пользователем значения. При нажатии «OK» возвращается введенное значение, в случае «Cancel» возвращается значение *null*.

Синтаксис: *prompt("вопрос/запрос","значение по умолчанию");*

Ниже приводится код веб-страницы, в которой пользователь имеет возможность выбрать цвет текста с помощью диалогового элемента

```
<html>
 <body>
   // здесь будет отображаться текст
   <div id="c" style="color:blue">Вы выбрали цвет текста: черный</div>
   <script language="JavaScript">
     // пользователь выбирает цвет текста
     var tcolor = prompt("Выберите цвет текста: red, blue, green, yellow, black","black");
     // задается текст
     document.getElementById("c").innerHTML = "Вы выбрали цвет текста: " + tcolor;
     // задается цвет текста
     document.getElementById("c").style.color = tcolor;
   </script>
 </body>
</html>
```

Общая постановка задачи

1. Описать структуру XML-документа с расписанием занятий с помощью DTD-схемы.

Создайте XML-документ *timetable.xml* со следующим содержимым:

```
<?xml version="1.0"?>
<timetable>
   <day dayOfWeek="Monday">
     <lesson type="practical">
         <timeFrom>08.00</timeFrom>
         <timeTo>09.30</timeTo>
         <subject>Deutsch</subject>
         <teacher>Borisova</teacher>
         <room>216</room>
     </lesson>
     <lesson type="lecture">
         <timeFrom>09.40</timeFrom>
         <timeTo>11.10</timeTo>
         <subject>SAP Administration</subject>
```

```
            <teacher>Egorov</teacher>
            <room>384</room>
        </lesson>
        <lesson type="practical">
            <timeFrom>11.20</timeFrom>
            <timeTo>12.50</timeTo>
            <subject>SAP Administration</subject>
            <teacher>Petrov</teacher>
            <room>384</room>
        </lesson>
    </day>
</timetable>
```

Загрузите этот файл в браузере. Обратите внимание на иерархическую структуру документа.

Добавьте в начало файла внутреннюю DTD-схему:

```
<?xml version="1.0"?>
<!DOCTYPE timetable[
<!ELEMENT timetable (day*)>
<!ELEMENT day (lesson+)>
<!ELEMENT lesson (timeFrom, timeTo, subject, teacher, room?)>
<!ELEMENT timeFrom (#PCDATA)>
<!ELEMENT timeTo (#PCDATA)>
<!ELEMENT subject (#PCDATA)>
<!ELEMENT teacher (#PCDATA)>
<!ELEMENT room (#PCDATA)>
<!ATTLIST day
        dayOfWeek CDATA #REQUIRED>
<!ATTLIST lesson
        type CDATA #REQUIRED>
]>
```

Сохраните файл и проверьте его на правильность с помощью программы-валидатора, например, по адресу: *http://validator.w3.org*. Попробуйте удалить какой-либо элемент или атрибут, проверьте документ снова с помощью валидатора.

Вынесите схему во внешний файл с размещением на него ссылки внутри XML-документа. Проверьте, как будет загружаться XML-документ.

2. Описать структуру XML-документа с расписанием занятий с помощью XML Schema.

Создайте XML-документ *timetable2.xml* со следующим содержимым:

```
<?xml version="1.0"?>
<timetable xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="timetable.xsd">
    <day dayOfWeek="Monday">
      <lesson type="practical">
          <timeFrom>08.00</timeFrom>
```

```
            <timeTo>09.30</timeTo>
            <subject>Deutsch</subject>
            <teacher>Borisova</teacher>
            <room>216</room>
        </lesson>
        <lesson type="lecture">
            <timeFrom>09.40</timeFrom>
            <timeTo>11.10</timeTo>
            <subject>SAP Administration</subject>
            <teacher>Egorov</teacher>
            <room>384</room>
        </lesson>
        <lesson type="practical">
            <timeFrom>11.20</timeFrom>
            <timeTo>12.50</timeTo>
            <subject>SAP Administration</subject>
            <teacher>Petrov</teacher>
            <room>384</room>
        </lesson>
    </day>
</timetable>
```

Создайте файл *timetable.xsd*, содержащий XML Schema:

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name = "timetable">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref = "day" minOccurs = "0" maxOccurs =
                "unbounded"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
    <xs:element name = "day">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref = "lesson" maxOccurs =
                "unbounded"/>
            </xs:sequence>
            <xs:attribute name = "dayofWeek" use = "required" type =
            "xs:string"/>
        </xs:complexType>
    </xs:element>
    <xs:element name = "lesson">
        <xs:complexType>
            <xs:sequence>
```

```
                    <xs:element ref = "timeFrom"/>
                    <xs:element ref = "timeTo"/>
                    <xs:element ref = "subject"/>
                    <xs:element ref = "teacher"/>
                    <xs:element ref = "room" minOccurs = "0"/>
                </xs:sequence>
            </xs:complexType>
        </xs:element>
        <xs:element name = "timeFrom" type = "xs:string"/>
        <xs:element name = "timeTo" type = "xs:string"/>
        <xs:element name = "subject" type = "xs:string"/>
        <xs:element name = "teacher" type = "xs:string"/>
        <xs:element name = "room" type = "xs:string"/>
    </xs:schema>
```

Проверьте схему на правильность с помощью программы-валидатора, например, по адресу: http://www.utilities-online.info/xsdvalidation.

4. Опишите структуру документа с экзаменационной ведомостью с помощью DTD схемы и XML Schema:

```
<gradeReport id="120851">
<date>10-06-2008</date>
<subject>Computer Science Fundamentals</subject>
<examiner>prof.Litvinov</examiner>
  <gradeList>
      <gradeRecord id="1">
        <student>Ivanov</student>
        <grade>4</grade>
      </gradeRecord>
      <gradeRecord id="2">
        <student>Petrov</student>
        <grade>3</grade>
      </gradeRecord>
      <gradeRecord id="3">
        <student>Sidorov</student>
        <grade>5</grade>
      </gradeRecord>
  </gradeList>
</gradeReport>
```

Проверьте обе схемы с помощью соответствующих программ-валидаторов.

5. Написать сценарий создания таблицы случайно выбранных цветов. Взяв за основу сценарий построения таблицы умножения, постройте таблицу случайно выбранных цветов. Цвет ячейки таблицы задается с помощью атрибута *bgcolor*. Цвет ячейки описывается в рамках трехкомпонентной модели RGB, например: *<td bgcolor=”#c0a145”>*. Для генерации каждой

компоненты можно использовать генератор случайных чисел с помощью методов объекта *Math* и преобразование в шестнадцатиричный формат:

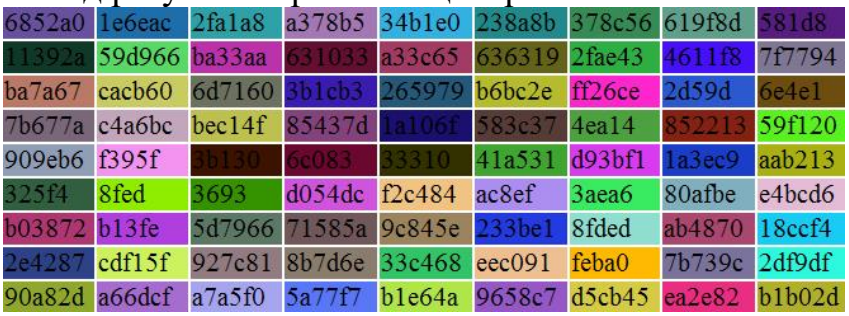
*color = Math.round(255.0*Math.random());*

r = color.toString(16);

Результирующий цвет образуется путем конкатенации компонентов:

color = r + g + b;

Примерный вид результата работы сценария:



6. Поместите разработанные схемы и сценарий на FTP-сервере. Проверить работу сценария, создав страницу с произвольной таблицей цветов.

7. Составить электронный отчет о проделанной работе. На титульном листе отчета указать URL для запуска разработанной программы на выполнение.